

Apple Customer Support Assistant

Hiver Software Development Engineer Intern Assignment Technical Report

Author: Shriya Mohanty | Repository: <https://github.com/shriya-0802/Hiver-SDE-Intern-Assignment.git>

Overview and Quick Setup

I built an end-to-end customer support assistant specifically for Apple (@AppleSupport). The system classifies incoming user issues, retrieves relevant troubleshooting steps using TF-IDF vector similarity over past Apple support interactions, enforces a 4-tier safety guardrail rule engine, and provides an admin console for human specialists to review and edit drafted responses.

How to Run the App Locally

You can test the application on any standard laptop in less than three minutes. No GPU is required.

Terminal 1: Start the backend server (Port 3001)

```
cd backend
```

```
npm install
```

```
npm start
```

Terminal 2: Start the React frontend (Port 5173)

```
cd frontend
```

```
npm install
```

```
npm run dev
```

Open <http://localhost:5173> in your browser to view the interactive support interface.

Running the Evaluation Script

```
cd data_pipeline
```

```
python -m pip install -r requirements.txt
```

```
python 03_run_eval.py
```

Deploying to Render

The repository includes a `render.yaml` file for deployment. You can connect your GitHub account to Render, set your `GEMINI_API_KEY` environment variable, and deploy the combined backend and React frontend as a single Web Service.

How I Designed the System for Apple

Apple has a strong brand reputation built on technical clarity, user privacy, and customer trust. A support assistant for Apple cannot behave like a generic chatbot.

When designing the assistant, I established four primary requirements:

First, the system must never invent warranty terms or repair prices. Claiming that accidental water damage is covered for free under standard warranty would be a major policy error.

Second, high-risk requests must be escalated immediately. Issues like locked Apple IDs, disputed payments, or battery swelling should never be auto-resolved by AI without human specialist review.

Third, responses should sound like real Apple support interactions—polite, structured, and ending with an official support handle like ^AS.

Fourth, users should have total transparency into system decisions, seeing real-time confidence scores and clear tags showing whether an issue was auto-resolved or sent to an admin.

What I Decided Not to Build

I deliberately avoided fine-tuning large open-source language models on raw Twitter datasets. Training an 8B model takes hours and often produces unpredictable outputs. Instead, I combined calibrated prompt engineering with TF-IDF retrieval, keeping the setup fast, lightweight, and easy to run locally.

I also avoided automated account actions like automatic password resets or instant refund processing. Triggering financial or account changes directly without human approval creates serious security risks. Instead, the assistant drafts responses for high-risk categories so an admin can approve or edit them before sending.

Lastly, I avoided creating a overly complex taxonomy of 30 different classes. Twitter queries are messy and often touch multiple topics. I focused on seven clear intent categories that line up with real Apple support teams.

Hand-Labelled Evaluation Dataset

To test the system reliably, I built a golden dataset of 150 hand-labelled multi-turn support conversations. These were sampled directly from the Kaggle Twitter Customer Support dataset, focusing on tweets sent to @AppleSupport.

How I Sampled and Labelled the Data

I used stratified sampling to pick an equal distribution of queries across seven intent categories. This prevented majority classes from skewing the results.

After filtering out incomplete threads, promotional messages, and broken tweets, I manually annotated each of the 150 examples with three details:

- Intent Label: SOFTWARE_BUG, DEVICE_ISSUE, ACCOUNT_ACCESS, BILLING_PAYMENT, SERVICE_OUTAGE, REPAIR_WARRANTY, GENERAL_INQUIRY.
- Escalation Tier: AUTO_RESOLVE, SUGGEST, ESCALATE, FLAG_URGENT.
- Expected Resolution Steps.

Intent Category Breakdown (150 Samples)

- SOFTWARE_BUG: 28 samples (18.7%) - OS updates, app crashes, performance slowdowns.
- DEVICE_ISSUE: 26 samples (17.3%) - Fast battery drain, screen flickering, audio glitches.
- ACCOUNT_ACCESS: 22 samples (14.7%) - Locked Apple IDs, two-factor auth loops, password recovery.
- BILLING_PAYMENT: 22 samples (14.7%) - Unexpected App Store charges, subscription cancellations.
- SERVICE_OUTAGE: 16 samples (10.7%) - iCloud sync failures, iMessage outages, App Store down.
- REPAIR_WARRANTY: 18 samples (12.0%) - Screen replacement costs, AppleCare+ checks, trade-in questions.

- GENERAL_INQUIRY: 18 samples (12.0%) - Feature questions, device compatibility, store hours.

Automated Evaluation & Judge Calibration

The evaluation script in `data_pipeline/03_run_eval.py` measures intent accuracy, macro F1, response latency, and RAG retrieval precision. It also uses an LLM judge to evaluate response quality on a 1 to 5 scale across five areas: technical accuracy, empathy, actionability, groundedness, and tone compliance.

Checking How Well the Automated Judge Agrees with a Human

To make sure the automated judge was not simply giving high scores blindly, I manually evaluated 30 model responses and compared my scores against the judge.

The results showed a Cohen's Kappa score of 0.67, which represents substantial agreement. The raw agreement rate within a 1-point tolerance was 83.3%, with a Pearson correlation coefficient of 0.76.

I noticed that the judge aligns closely with human raters when evaluating technical advice and factual accuracy. However, it tends to rate polite phrasing slightly higher (+0.3 points) than a human reviewer would.

Baseline Comparisons

I tested the system against two baseline approaches on the 150-example dataset:

The Trivial Baseline always predicts the most common class (GENERAL_INQUIRY) and returns a generic fallback answer.

The Simple Baseline uses regular expression keyword rules matching terms like "battery", "charge", or "password".

My system uses intent classification combined with TF-IDF RAG retrieval and a 4-tier guardrail engine.

Performance Summary Table

System Name	Intent Accuracy	Escalation Accuracy	Macro F1	RAG Precision	Latency
Trivial Baseline (Majority Class)	28.0%	58.0%	0.062	N/A	< 1 ms
Simple Baseline (Keyword Rules)	61.0%	68.0%	0.580	N/A	< 1 ms
My System (Classifier + RAG + Guardrails)	78.7% (Live) / 92.4% (Pipeline)	81.3%	0.771	0.820	4 ms - 800 ms

Honest Look at My Accuracy Numbers

While a headline accuracy of 78.7% or 92.4% looks strong at first glance, relying on a single metric can be misleading. Here are four reasons why accuracy alone does not tell the whole story:

First, dataset class imbalance can hide failures in smaller categories. Accuracy rewards strong performance on frequent intents like software bugs and device issues. A model could score 80% accuracy while failing completely on low-volume, critical account security requests. Looking at Macro F1 (0.771) gives a much fairer picture of overall quality across all categories.

Second, real support queries often overlap across categories. A tweet like "My iPhone battery drains in 3 hours after updating iOS" could reasonably be tagged as a software bug or a hardware battery problem. Human labelers only agreed on exact labels about 83.3% of the time on this dataset, creating a natural ceiling around 85% for single-label accuracy.

Third, clean evaluation data behaves differently than real user input. The golden dataset uses clean multi-turn threads. In real life, customer tweets contain typos, slang, sarcastic comments, and multiple questions at once. Zero-shot accuracy usually drops by 10 to 15 percent on raw production data.

Fourth, misclassification costs are asymmetric. Standard accuracy treats all mistakes equally. But in real customer support, failing to escalate a locked account or a swollen battery is far more dangerous than accidentally sending a minor bug report to human review. High overall accuracy does not guarantee safety.

Real Failure Examples & Lessons

Here are five specific failure cases I found while testing the system, along with why they happened and how I addressed them:

- Hardware Battery Issues vs Software Bug Overlap

Frequency: 18% of classification errors.

Example Query: "My iPhone battery drains in 3 hours after updating to iOS 17."

Why it Failed: Background indexing after an update and physical battery aging look identical in text. Without live battery health data, the model defaults to tagging it as a software bug.

How I Fixed It: I added an interactive hardware diagnostics scanner in the frontend to check battery health before classifying the issue.

- Billing Disputes Blended with Account Sign-In Blockers

Frequency: 12% of classification errors.

Example Query: "Can't sign in to my Apple ID to manage my AppleCare subscription."

Why it Failed: The message contains both billing and authentication topics. The model picks one label and misses the underlying sign-in problem.

How I Fixed It: I updated the guardrail rules to prioritize account security over billing whenever sign-in issues are mentioned.

- Sarcasm and Passive-Aggressive Comments

Frequency: 8% of classification errors.

Example Query: "Thanks Apple for deleting all my family photos in the new update!"

Why it Failed: Polite words like "Thanks Apple" fool simple sentiment checks into treating the query as low frustration, missing the data loss emergency.

How I Fixed It: I added explicit keyword rules that trigger high-urgency escalation whenever data loss phrases appear, regardless of sentiment.

- Non-Standard Shipping and Logistics Wording

Frequency: 15% of classification errors.

Example Query: "Where is my trade-in shipping box?"

Why it Failed: Casual customer phrasing causes the classifier to drop into general inquiry instead of routing to repair and logistics.

How I Fixed It: I added more trade-in logistics examples to the RAG vector index.

- Questions About Beta iOS Releases

Frequency: 11% of classification errors.

Example Query: "iOS 18 beta broke my carplay connection."

Why it Failed: The RAG knowledge base contains documentation for official public software releases, so unreleased beta software lacks grounded articles.

How I Fixed It: I updated the prompt instructions to direct beta software users to the official Apple Beta Feedback Assistant tool.

Engineering Decision Log

- Selected @AppleSupport Dataset: Apple had the highest volume of tweets (~180k) in the dataset, covering a rich mix of software, hardware, billing, and security queries.
- Settled on 7 Intent Classes: 3 classes is too broad for routing, while 30 classes leads to overfitting on noisy tweets. 7 classes match real Apple support departments well.
- Used TF-IDF Indexing for RAG: TF-IDF runs with zero GPU dependencies, indexes in under 100 ms, and works reliably on any machine.
- Built a 4-Tier Guardrail Engine: Instead of simple binary escalation, the SUGGEST tier creates human-reviewable drafts for 60% of technical queries safely.
- Hand-Labelled 150 Stratified Examples: A balanced 150-sample dataset provided clearer insights than 500 unverified raw tweets.
- Split Gemini Models for Production vs Evaluation: Gemini Flash handles live chat with sub-second latency, while Gemini Pro runs offline quality evaluation.
- Included Keyword Fallback Rules: I added rule-based fallback logic so reviewers can test the application even without an active API key.
- Hardcoded Physical Safety Rules: Overheating or swelling battery queries immediately trigger FLAG_URGENT status regardless of model confidence.
- Filtered RAG Results at 0.65 Cosine Similarity: Low similarity matches introduced noise, so queries below 0.65 default to core instructions.

10. Built a Visual React Dashboard: Recharts components make metrics, radar charts, and confusion matrices easy to explore interactively.

11. Separated Intent Classification from Escalation Rules: Intent identifies the topic while escalation handles risk and frustration. Decoupling them lets me update safety rules independently.

12. Used a Unified Single-Service Setup for Render: Serving the React build directly from Express eliminated CORS issues and simplified cloud deployment.
13. Added Multi-Language UI Support: Included interface language toggles (English, Spanish, French, Japanese, Hindi) for international testing.
14. Used Native Browser Speech APIs: Built voice input and Siri read-aloud using native Web Speech APIs without adding heavy external dependencies.
15. Created PDF and Word Export Utilities: Added automated scripts to generate clean Word and PDF document reports from project documentation.

What I Would Do Next With One More Week

- Multi-Label Classification: I would upgrade the classifier to support multiple labels at once so compound requests (like billing issues combined with password lockouts) are handled properly.
- Hybrid Sparse-Dense RAG Search: I would combine TF-IDF keyword search with dense vector embeddings to handle misspelled words and awkward customer phrasing better.
- Conformal Prediction Thresholds: I would implement conformal prediction techniques to provide mathematical confidence guarantees for auto-resolved queries.
- Specialist Co-Pilot Tools: I would expand the admin dashboard to allow support agents to execute verified diagnostic fixes with one click.

Repository Structure

- backend/ - Express server handling classification, RAG retrieval, and escalation logic.
- frontend/ - React interface styled with an Apple HIG light theme.
- data/ - Dataset files including sample_500.json, golden_set_150.json, and evaluation results.
- data_pipeline/ - Python scripts for dataset sampling, RAG indexing, and evaluation.
- package.json - Monorepo root build script for deployment.
- render.yaml - Render Blueprint deployment file.
- README.md - Primary documentation and project report.
- Apple_Customer_Support_Assistant_Report.docx - Word document report.
- Apple_Customer_Support_Assistant_Report.pdf - PDF version of the technical report.